Draft version June 12, 2026
Typeset using LaTeX **twocolumn** style in AASTeX7.0.1

Graph Database Solution for Higher Order Spatial Statistics in the Era of Big Data: GPU Acceleration and Out-of-Core N-point Functions

Cristiano G. Sabiu[1]

[1] *Department of Physics, University of Seoul, Seoul 02504, Republic of Korea*

ABSTRACT

We present GRAMSCI (GRAph Made Statistics for Cosmological Information), an algorithm and open-source Fortran code for the fast computation of the general N-point spatial correlation function of any discrete point set embedded in $\mathbb{R}^n$. Using KD-trees and a graph database, we count all possible N-tuples in binned configurations within a given length scale.

Version 2 update. We describe several additions to the original code. First, we replace the binary-search inner loop — which cost $O(m \log m)$ per hub–spoke pair, where m is the mean neighbour count — with a *merge-walk*: a two-pointer sorted-set intersection that exploits the pre-sorted adjacency lists and reduces the inner loop to $O(m)$. Second, we implement a parity-decomposed 4PCF that separates the signal into even and odd channels, enabling direct tests of parity violation in the galaxy distribution. Third, we provide a Python interface so the Fortran engine can be called directly from NumPy arrays. Finally, and principally, we present a GPU port of the full query engine (OpenACC): the 3PCF, equilateral 3PCF, 4PCF, and parity-decomposed 4PCF kernels run on a single consumer GPU with measured speedups of 2.6× (3PCF) to 9× (4PCF) over a 64-thread CPU node, and an out-of-core tiling scheme allows graphs far exceeding device memory — we measure a 9×10^9-edge BAO-scale 3PCF on a 24 GB card with ∼20% overhead. We validate the code against its CPU reference, against analytic injection tests, and on Quijote N-body simulations, and demonstrate BAO-scale applications on the DESI DR1 LRG sample: the 3PCF and connected 4PCF are measured in two redshift bins and compared against the EZmock ensemble, with the acoustic feature visible in the three-point function and reproduced by the mocks. The code is available at https://github.com/csabiu/Gramsci.

Keywords: cosmology: large-scale structure — methods: statistical — methods: numerical — galaxies: statistics

1. INTRODUCTION

The spatial distribution of galaxies encodes information about the primordial density field and its subsequent gravitational evolution. The two-point correlation function (2PCF) has been the workhorse of this programme (e.g. L. Anderson & et al. 2014), but higher-order statistics carry complementary information that is inaccessible to the power spectrum. The 3-point correlation function (3PCF), measured in galaxy catalogues since P. J. E. Peebles & E. J. Groth (1975), probes the bispectrum, which constrains primordial non-Gaussianity and galaxy bias (E. Sefusatti et al. 2006; E. Gaztañaga & R. Scoccimarro 2005), and has yielded a detection of the baryon acoustic peak in BOSS (Z. Slepian & et al. 2017). The connected 4-point correlation function (4PCF) is related to the primordial trispectrum and provides additional leverage on inflationary models (E. Sefusatti & E. Komatsu 2007).

Computing N-point functions from large catalogues is expensive. A brute-force approach requires $O(N^k)$ operations for a k-point function. Tree-based methods reduce this to $O(Nm^{k-1})$, where m is the mean number of neighbours within the maximum scale $r_{\max}$, but the m^{k-1} factor still grows rapidly.

Several codes are publicly available. TreeCorr (M. Jarvis et al. 2004) provides optimised 2PCF and 3PCF estimators for projected and three-dimensional catalogues. Corrfunc (M. Sinha & L. H. Garrison 2020) uses SIMD vectorisation (AVX2/AVX-512) for maximum 2PCF throughput. A complementary algorithmic line, initiated by Z. Slepian & D. J. Eisenstein (2015) and accelerated with FFTs by Z. Slepian & D. J. Eisenstein (2016), expands the correlators in a spherical-harmonic basis: Encore (O. H. Philcox & Z. Slepian

2022) extends this approach to the N-PCF, replacing direct enumeration with matrix products; this achieves favourable scaling at high neighbour density but does not directly yield a configuration-space connected 4PCF with parity decomposition.

GRAMSCI (C. G. Sabiu et al. 2019) uses a graph-database approach: it builds a compact sorted adjacency structure once from a KD-tree, then queries it for any combination of 2-, 3-, and 4-point statistics without revisiting the original coordinates. This paper extends the original work in four directions (Section 3): an $O(m)$ merge-walk inner kernel (Section 3.1), a parity-decomposed 4PCF (Section 3.2), a Python interface (Section 3.3), and a GPU port of the complete query engine with out-of-core support for graphs exceeding device memory (Section 3.4).

2. THE GRAMSCI ALGORITHM

We summarise the algorithm here; full details are in C. G. Sabiu et al. (2019).

2.1. *Graph construction*

Given a catalogue of N objects, GRAMSCI builds a KD-tree and finds, for each object i, all neighbours within $r_{\max}$. These are stored as a sorted adjacency list:

$$\mathcal{N}(i) = \{j_1, j_2, \ldots, j_{m_i}\}, \quad j_1 < j_2 < \cdots < j_{m_i}. \tag{1}$$

Alongside each neighbour index, the radial-bin index (an integer byte) and optionally a direction-pixel byte (for the parity 4PCF) are stored. Counts are accumulated with signed, normalised data-minus-random weights in the style of the S. D. Landy & A. S. Szalay (1993) and I. Szapudi & A. S. Szalay (1998) estimators. The KD-tree and raw coordinates are then discarded; all subsequent NPCF queries operate entirely on the adjacency lists.

The total memory footprint is $O(Nm)$ rather than $O(N^2)$. The sorted order in Equation (1) is a key invariant exploited by the merge-walk kernel and the GPU binary-search kernels alike.

2.2. *Triangle and tetrahedron enumeration*

For the 3PCF, GRAMSCI uses a hub-and-spoke loop: for each hub i and each neighbour $j \in \mathcal{N}(i)$, it checks whether a second neighbour $k \in \mathcal{N}(i)$ with $k > j$ is also in $\mathcal{N}(j)$. If so, the triangle (i, j, k) is counted in the appropriate (r_{ij}, r_{ik}, r_{jk}) bin. The 4PCF extends this to tetrahedra (i, j_1, j_2, j_3) where every pair is connected. The original membership test was a binary search on $\mathcal{N}(j)$, costing $O(\log m)$ per candidate.

3. NEW CONTRIBUTIONS

3.1. *Merge-walk: $O(m)$ sorted-set intersection*

Since the hub tail $\{k \in \mathcal{N}(i) : k > j\}$ and $\mathcal{N}(j)$ are both sorted by node index (Equation 1), their intersection can be found in a single linear pass with two pointers a and b:

- a walks the hub tail, starting at the entry after j.
- b walks $\mathcal{N}(j)$ from its beginning.

At each step:

- $\mathcal{N}(i)[a] = \mathcal{N}(j)[b]$: triangle found; accumulate the (r_{ij}, r_{ik}, r_{jk}) bin contribution; advance both.
- $\mathcal{N}(i)[a] < \mathcal{N}(j)[b]$: advance a.
- otherwise: advance b.

Each list is visited at most once, giving $O(m_i + m_j)$ cost per spoke — an $O(\log m)$ improvement over binary search.

3.1.1. *Three-way merge for the 4PCF*

For the 4PCF, once the k2 walk has located a second spoke j_2, the k3 loop must find a third spoke j_3 that appears simultaneously in the hub tail beyond j_2, in $\mathcal{N}(j_1)$, and in $\mathcal{N}(j_2)$. We generalise the two-pointer walk to three pointers α, β, γ:

- α: hub tail beyond j_2.
- β: $\mathcal{N}(j_1)$, starting at the position just after j_2 (reused from the k2 walk, so no rescanning).
- γ: $\mathcal{N}(j_2)$, pre-advanced past j_2's own index (entries $\le j_2$ can never match the hub tail).

Let $h_\alpha, h_\beta, h_\gamma$ be the current candidate node indices. If all three are equal, we record the tetrahedron and advance all three. Otherwise every pointer at the current minimum is advanced:

$$\begin{aligned} h_\alpha \le h_\beta \text{ and } h_\alpha \le h_\gamma: &\quad \alpha \leftarrow \alpha + 1 \\ h_\beta \le h_\alpha \text{ and } h_\beta \le h_\gamma: &\quad \beta \leftarrow \beta + 1 \\ h_\gamma \le h_\alpha \text{ and } h_\gamma \le h_\beta: &\quad \gamma \leftarrow \gamma + 1 \end{aligned} \tag{2}$$

(Multiple conditions can be true simultaneously when two minima are tied; in that case multiple pointers are advanced in the same step.) This visits each list at most once, giving $O(m)$ cost per (i, j_1, j_2) triple compared with two $O(\log m)$ binary searches in the original code.

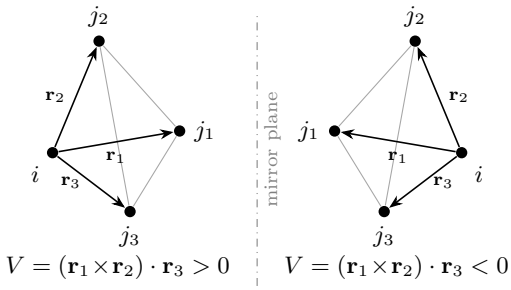

Figure 1. Chirality of a galaxy 4-tuple. A tetrahedron (i, j_1, j_2, j_3) is the smallest configuration of points with a handedness, fixed by the sign of the signed volume V (Equation 3). Its mirror image (right) has identical pair separations — the same six edge bins — but opposite V, and no rotation maps one onto the other. GRAMSCI accumulates the two handednesses with opposite signs in the parity-odd channel ζ^-, which therefore vanishes for any parity-symmetric field.

3.2. *Parity-decomposed 4PCF*

A standard 4PCF estimator gives one number per tetrahedron configuration. GRAMSCI v2 adds a parity decomposition that separates the signal into parity-even (ζ^+) and parity-odd (ζ^-) channels. For each tetrahedron (i, j_1, j_2, j_3), the handedness is determined by the signed volume

$$V = (\mathbf{r}_{ij_1} \times \mathbf{r}_{ij_2}) \cdot \mathbf{r}_{ij_3}. \tag{3}$$

Figure 1 illustrates the underlying geometry: a tetrahedron of galaxies is the smallest configuration with a handedness, and its mirror image cannot be rotated back onto the original. The sign of V distinguishes the two. The direction of each spoke is also quantised into a pixel on the unit sphere (stored as a single byte per edge at graph-construction time with ∼17% memory overhead), enabling efficient grouping by parity at query time without retaining the original coordinates.

Tetrahedra whose pixelised direction vectors give a mathematically zero signed volume (repeated pixels, coplanar pixel triples) have no defined chirality. These are common — repeated pixels alone account for a few per cent of all tetrahedra at typical pixelisations, comparable to the amplitude of the odd channel itself — and assigning them a sign from the floating-point residue of Equation (3) would contaminate ζ^- with compiler- and hardware-dependent noise. GRAMSCI therefore assigns degenerate tetrahedra ($|V| < 10^{-9}$) zero weight in the odd channel; the even channel is unaffected.

The parity-odd 4PCF ζ^- vanishes for any statistically parity-symmetric field; a non-zero measurement signals parity violation (R. N. Cahn et al. 2023). Searches for such a signal in BOSS galaxy catalogues (J. Hou et al. 2023; O. H. E. Philcox 2022) — and most recently in DESI (Z. Slepian & et al. 2025) — have generated significant interest, and independent configuration-space measurements provide a direct cross-check on spherical-harmonic-based analyses.

3.3. *Python interface*

A lightweight Python package (requiring only NumPy) wraps the compiled Fortran binary. Input/output is handled automatically through a temporary directory; the user supplies NumPy arrays and receives structured result objects:

Listing 1. Minimal Python usage.

```
import gramsci, numpy as np

# pos: (N,3) float64,  weights: (N,) float64
r3 = gramsci.compute_3pcf(
        pos_data, w_data, pos_rand, w_rand,
        rmin=1.0, rmax=30.0, nbins=6)
print(r3.zeta)   # shape (n_configs,)
print(r3.NNN, r3.r1, r3.r2, r3.r3)

r4 = gramsci.compute_4pcf(
        pos_data, w_data, pos_rand, w_rand,
        rmin=1.0, rmax=30.0, nbins=3,
        parity=True)
print(r4.zeta_even)  # parity-even 4PCF
print(r4.zeta_odd)   # parity-odd 4PCF
```

The package is installable with `pip install -e .` from the `python/` subdirectory of the repository.

3.4. *GPU implementation*

The dominant cost in every GRAMSCI query is the enumeration loop over hub–neighbour tuples — millions of independent hubs, each performing $O(m^2)$ to $O(m^3)$ work on small sorted integer lists. This structure maps naturally onto a GPU, and we have ported the complete query engine (3PCF, equilateral 3PCF, 4PCF, and parity 4PCF) to OpenACC, compiled with NVFORTRAN. The GPU binary, `gramsci_gpu`, accepts the same command-line options and produces identical output formats to the CPU code. Graph construction remains on the CPU (OpenMP) and is typically a small fraction of the total runtime.

3.4.1. *Data layout*

The CPU code stores adjacency lists as per-node allocatable arrays, a structure that cannot be mapped to device memory. The GPU path flattens the graph into a compressed-sparse-row (CSR) form: a 64-bit offset array `ptr`$(1{:}N{+}1)$ and flat edge arrays holding the neighbour index (32-bit) and radial-bin index (8-bit) — 5 bytes per edge, plus a direction-pixel byte for parity queries. Edge counts routinely exceed 2^{31} for BAO-scale work (Section 4), so all edge indexing is 64-bit. During the flattening, each jagged row is freed as it is copied (with

the freed pages explicitly returned to the operating system), holding peak host memory to approximately one live copy of the graph.

3.4.2. *Kernel design*

Three design rules, found empirically, determine GPU performance (Appendix A records the failure modes of the alternatives):

One thread block per hub, vector lanes on the inner loop. Each hub is assigned to one OpenACC gang (CUDA thread block); the innermost neighbour loop is spread across the gang's vector lanes. For a fixed hub–spoke pair, all lanes then search or read the *same* adjacency list, so a warp's memory traffic is confined to one small, cache-resident segment. Letting the compiler choose the mapping instead places a different hub on every vector lane, producing fully divergent memory access; we measured this naïve mapping to be 6× slower than the 64-thread CPU code, versus 2.6× faster for the explicit mapping.

Per-hub scratch via gang slots. The 4PCF kernels precompute, per hub, a local connectivity matrix $\texttt{lmat}(k_2, k_1)$ holding the bin index of the edge between neighbours $k_1 < k_2$, reducing the innermost tetrahedron loop to $O(1)$ array lookups: $C(m, 2)$ searches replace the original $3\,C(m, 3)$. This matrix (up to several hundred kilobytes for dense graphs) cannot be a private array — OpenACC privatises arrays per vector lane — so each gang owns a slice of a global scratch array indexed by an explicit gang-slot loop. The scratch extent is sized at runtime from the longest adjacency row of the actual graph.

Atomic accumulation into slot-strided partials. Vector lanes within a gang race on shared accumulators, and compiler-generated array reductions across nested gang/vector loops proved unreliable or slow. Counts are instead accumulated with hardware floating-point atomics into per-slot partial arrays (one slot per gang, or hash-strided over ∼4k slots), which are summed on the host. Atomic contention is negligible because concurrently resident gangs own distinct slots.

3.4.3. *Out-of-core operation*

At BAO scales a survey-sized graph exceeds any single GPU's memory: the DESI DR1 LRG North+South sample at $r_{\max} = 150\,h^{-1}$Mpc produces 9.0×10^9 edges (45 GB of CSR). The query kernels therefore support a chunked mode, selected automatically at runtime when the graph exceeds the free device memory.

The kernel needs two kinds of row access: the hub's own row, and the row of any neighbour being searched. The edge arrays are split into W row-aligned windows, and the query runs as a tile loop over (hub-window, search-window) combinations with only the active windows resident on the device. A triangle at hub i with spoke pair (j_1, j_2) is processed exactly once — in the tile whose hub window holds i's row and whose search window holds j_1's row — and tiles skip foreign spokes with a single integer comparison. For the 4PCF kernels, whose connectivity matrix draws on two different neighbour rows, tiles run over *pairs* of search windows ($c_1 \le c_2$); the sorted adjacency invariant guarantees each tetrahedron lands in exactly one tile. Window sizes are derived at runtime from free device memory and can be overridden through an environment variable, which also provides a convenient way to exercise the chunked code paths on small test data.

The measured cost of chunking is modest: the 9.0×10^9-edge DESI measurement above ran in 28 minutes on a 24 GB RTX 3090 Ti, roughly 20% slower than the extrapolated cost of a hypothetical all-resident run, the difference being PCIe re-transfers of the windows.

4. PERFORMANCE BENCHMARKS

4.1. *Test catalogue*

All CPU benchmarks used a Gaussian mock with $N_g = 207\,246$ galaxies and $N_r = 500\,000$ randoms in a periodic box. Timings are wall-clock seconds on a single node with 4 OpenMP threads, divided by thread count to give effective single-core seconds for fair comparison. Radial bins are equally spaced from $r_{\min} = 1\,h^{-1}$Mpc to $r_{\max}$.

Benchmarks were run on a shared (but lightly loaded) workstation rather than a dedicated node. To quantify the resulting timing noise, several configurations were measured repeatedly at different times during the campaign: CPU timings reproduce to 0.6–1.1% and GPU timings to $\lesssim$5%, far below the effects reported here. [TODO: re-verify sentinel timings on a quiescent node before submission (analysis/benchmarks/verify_sentinels.py)]

All benchmarks in this paper compare implementations of the same estimator within GRAMSCI — identical binning, identical outputs, identical hardware — so every quoted speedup is directly attributable to the algorithmic or architectural change being tested. We deliberately make no cross-code timing claims: the publicly available N-point codes (Section 1) compute related but distinct quantities (projected versus three-dimensional statistics, harmonic-basis multipoles versus binned configuration-space functions), and timing comparisons across different output products are not meaningful.

4.2. *Merge-walk speedup*

Table 1 summarises the CPU results. The speedup grows with $r_{\max}$ because the mean neighbour count $m \propto r_{\max}^3$ and the savings per inner-loop step scale as $\log_2 m$. At $r_{\max} = 55\,h^{-1}$Mpc ($m \approx 700$) the 3PCF achieves 2×; at $r_{\max} = 40\,h^{-1}$Mpc the 4PCF achieves 3.8× because two independent binary searches were replaced by a single three-pointer walk.

4.3. *Three code generations*

Figure 2 shows the central benchmark of this paper: query time as a function of $r_{\max}$ for the three generations of the code — the v1 binary-search kernels, the v2 merge-walk, and the GPU port — all computing identical binned counts, with both CPU generations using all 64 threads. The merge-walk advantage measured at 4 threads (Table 1) persists at full thread count (3.2× for the 3PCF and 3.7× for the 4PCF at the largest common scales). The GPU curves carry a fixed ∼0.7 s offset (PCIe transfers and kernel launch), so the CPU wins below a crossover scale ($r_{\max} \approx 55\,h^{-1}$Mpc for the 3PCF, ≈35 h^{-1}Mpc for the 4PCF) beyond which the GPU pulls away along the same $r_{\max}^6$ and $r_{\max}^9$ power laws. At $r_{\max} = 70\,h^{-1}$Mpc the 4PCF lineage reads 1044 s → 280 s → 26 s: a cumulative 41× from the original published kernel to the GPU on the same node. Table 2 adds measurements on the DESI DR1 LRG sample (Section 5). In all cases the binned counts agree exactly with the CPU reference.

4.4. *Parallel scaling*

Figure 3 shows OpenMP strong scaling of the v2 CPU kernels from 1 to 64 threads, with the GPU time as a horizontal reference. Both kernels scale with 76% (3PCF) and 74% (4PCF) efficiency at 64 threads — the roll-off reflects the `reduction`-clause accumulator copies and dynamic-schedule load imbalance from the long tail of high-multiplicity hubs. On these problems the single GPU matches roughly 95–100 CPU cores of equivalent throughput.

4.5. *Density dependence*

Figure 4 isolates the dependence on sample density by subsampling the test catalogue (galaxies and randoms together) at fixed $r_{\max}$. Query time follows the expected $t \propto f^3$ for the 3PCF (hubs $\propto f$, pairs per hub $\propto f^2$) with a steeper $f^{3.3}$ for the 4PCF. The GPU–CPU crossover sits near $f \approx 0.4$: below it the fixed transfer cost dominates and the CPU is faster; above it the GPU advantage grows with density, reaching 2.6× (3PCF) and 6.1× (4PCF) at full density and continuing to widen for denser samples. Together with Figure 2 this defines the regime where the GPU pays off: roughly, whenever the CPU query exceeds a few seconds.

4.6. *Out-of-core overhead*

Figure 5 measures the cost of the window tiling of Section 3.4.3 directly, by forcing a fixed problem (3PCF at $r_{\max} = 80\,h^{-1}$Mpc, 1.9×10^8 edges) through W = 1–8 windows. Going out-of-core costs 12% once, after which the overhead is nearly flat (17% at $W = 8$): the re-staged windows and repeated hub scans grow only linearly in W while the dominant pair enumeration is partitioned, not repeated. This plateau is corroborated at 50× the scale by the DESI measurement of Table 2, where the 9.0×10^9-edge query at $W = 5$ ran ∼20% above the single-window extrapolation.

4.7. *Expected performance on other hardware*

All GPU timings in this paper were obtained on a single consumer card (NVIDIA RTX 3090 Ti: Ampere, 24 GB, ∼1 TB s^{-1} memory bandwidth). The query kernels are bound by memory access — sorted-list traversals, byte-wide bin loads, and table lookups — rather than by floating-point throughput: the only FP64 work is a few multiplications and one atomic addition per accepted tuple. Performance should therefore track *memory bandwidth* rather than FP64 capability, and the large FP64 advantage of data-centre GPUs is largely irrelevant here. On this basis we expect roughly 2× on A100-class hardware (2.0 TB s^{-1}) and 3–3.5× on H100-class hardware (3.4 TB s^{-1}), with the caveat that occupancy and atomic-throughput differences between architectures are not captured by this single-axis estimate. Device memory capacity changes behaviour qualitatively rather than quantitatively: an 80 GB card raises the single-pass ceiling to $\sim 1.5 \times 10^{10}$ edges — the largest measurement in this paper would fit without tiling — and eliminates the out-of-core overhead of Section 3.4.3, while NVLink-class host links shrink the transfer component of that overhead where tiling is still required. These are expectations, not measurements. Finally, the tile decomposition of Section 3.4.3 partitions work by hub window with no communication between tiles, so a multi-GPU extension — one hub window per device — is straightforward in principle; we leave its implementation to future work.

4.8. *Correctness verification*

Both implementations are run on the same catalogue and compared bin by bin. All four GPU kernels (3PCF, equilateral 3PCF, 4PCF, parity 4PCF) were validated against the CPU reference in both single-pass and chunked modes — eight combinations — with exact agreement of all binned counts. The regression test

Table 1. Wall-clock speedup of the merge-walk over binary search.

Kernel	$r_{\max}$ $[h^{-1}\mathrm{Mpc}]$	Bsearch [s]	Merge-walk [s]	Speedup
3PCF	30	6.6	4.5	1.46×
3PCF	40	34	21	1.65×
3PCF	55	223	109	2.04×
4PCF	30	61.6	18.8	3.27×
4PCF	40	666.7	174.1	3.83×
4PCF (parity)	30	69.2	24.8	2.79×

NOTE—Single-node, 4 OpenMP threads; times divided by thread count. The 4PCF gain is larger than the 3PCF gain because the original code had two binary searches in the innermost loop.

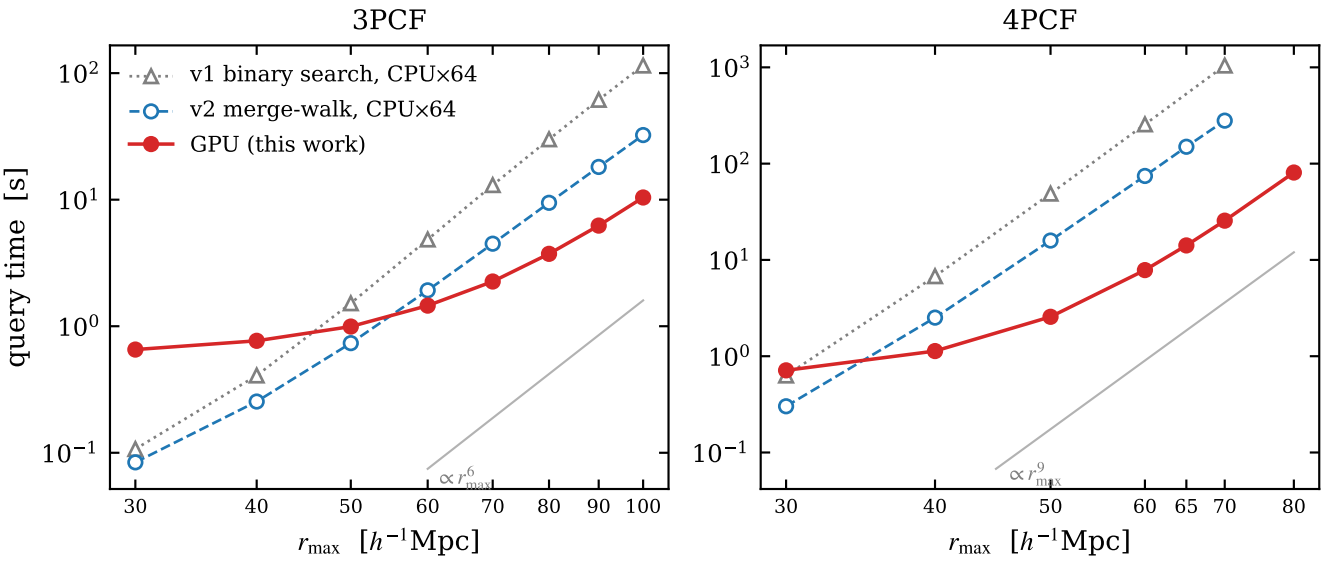

Figure 2. Query time versus $r_{\max}$ for the three generations of GRAMSCI: the original binary-search kernels (v1, grey triangles), the merge-walk (v2, open blue circles), and the GPU port (red), for the 3PCF (left) and 4PCF (right) on the test catalogue (0.7M points). Both CPU generations use all 64 threads. Grey guides show the expected $r_{\max}^6$ and $r_{\max}^9$ scalings of the pair- and triple-enumeration work. The GPU pays a fixed ∼0.7 s transfer cost and overtakes the CPU at $r_{\max} \approx 55$ (35) h^{-1}Mpc for the 3PCF (4PCF).

suite (included in the repository) checks isolated-pair 2PCF, regular-tetrahedra connected 4PCF, and chiral-tetrahedra parity 4PCF against known analytic values, and `tests/benchmark_3pcf.py` automates the CPU–GPU comparison.

4.9. *Validation of the parity estimator*

The parity-odd channel deserves its own end-to-end test, since it is the quantity for which subtle sign errors (Section 3.2) are most consequential. We perform a controlled signal-injection test: catalogues of $N_{\mathrm{struct}} = 500$ isolated scalene tetrahedra of fixed geometry are generated with a left-handed fraction x, embedded in uniform randoms. Each left-(right-)handed unit contributes +1 (−1) to the parity-odd count at the injected edge-bin configuration, and with normalised weights the count-level prediction is exact: NNNN$^-$ $N^4_{\mathrm{gal}} = N_L - N_R$, with mixed data–random terms cancelling in the odd channel in expectation. Figure 6 shows the recovered asymmetry against the injected value for $x = 0.5$–1: the estimator is unbiased at the 0.3% level across the full range, and

Table 2. GPU query times vs. a 64-thread CPU node (RTX 3090 Ti vs. 64 OpenMP threads).

Kernel	Catalogue	CPU×64 [s]	GPU [s]	Speedup
3PCF, $r_{\max}$=80	test (0.7M)	9.4	3.6	2.6×
4PCF, $r_{\max}$=50	test (0.7M)	15.7	2.9	5.5×
4PCF, $r_{\max}$=65	test (0.7M)	146.8	16.4	9.0×
3PCF, $r_{\max}$=120	DESI LRG (3.0M)	···	380	···
3PCF, $r_{\max}$=150[a]	DESI LRG (3.0M)	···	1674	···

[a] Out-of-core: 9.0×10^9 edges (45 GB CSR) on a 24 GB device, 5×5 window tiles.

Note—Query time only; graph construction (CPU, OpenMP) adds $\lesssim$ 10% for the GPU runs. [TODO: re-run CPU reference timings for the DESI rows on the 64-thread node for completeness]

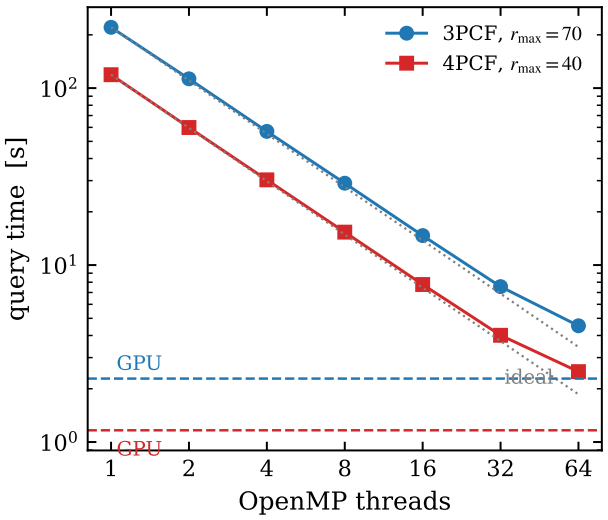

Figure 3. OpenMP strong scaling of the CPU query kernels (3PCF at $r_{\max}$=70 h^{-1}Mpc, 4PCF at $r_{\max}$=40 h^{-1}Mpc; dotted guides show ideal scaling). Dashed horizontal lines mark the single-GPU times for the same problems.

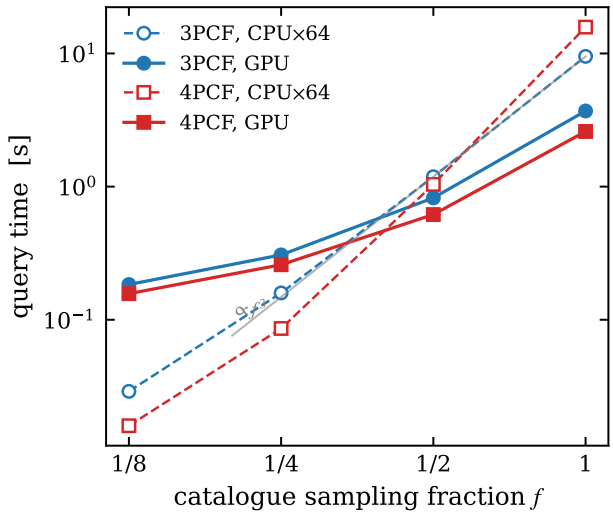

Figure 4. Query time versus catalogue sampling fraction f at fixed $r_{\max}$ (3PCF at 80 h^{-1}Mpc, 4PCF at 50 h^{-1}Mpc). The grey guide shows $t \propto f^3$. The GPU overtakes the 64-thread CPU near $f \approx 0.4$ and its advantage grows with density.

the parity-balanced case ($x = 0.5$) is consistent with zero. We note that the same test expressed as the ratio ζ^-/ζ^+ would show a ∼7% offset — not an estimator bias, but the constant mixed-term contribution to the even channel at this configuration, which cancels only in the odd channel; count-level comparisons are the appropriate validation target.

The complementary null test on cosmological density fields uses the parity-symmetric Quijote fiducial realisations (Section 6): gravity produces no chirality, so the parity-odd 4PCF measured on these boxes must be consistent with zero configuration by configuration. Across 66 realisations and 638 well-measured configurations ($r_{\max}$ = 65 h^{-1}Mpc, 5 radial bins) we find $\chi^2/\mathrm{dof} = 1.10$ for the mean parity-odd counts against zero, with 5.6% of configurations beyond 2σ (Gaussian expectation 4.6%). The appropriate reference value is not unity: with N_m realisations the per-configuration error estimates make the z-scores Student-t distributed, giving an expected $\chi^2/\mathrm{dof} = (N_\mathrm{m}-1)/(N_\mathrm{m}-3) = 1.03$, from which the measurement deviates by ∼1σ before accounting for inter-configuration correlations. The same statistic computed without the degenerate-volume rule

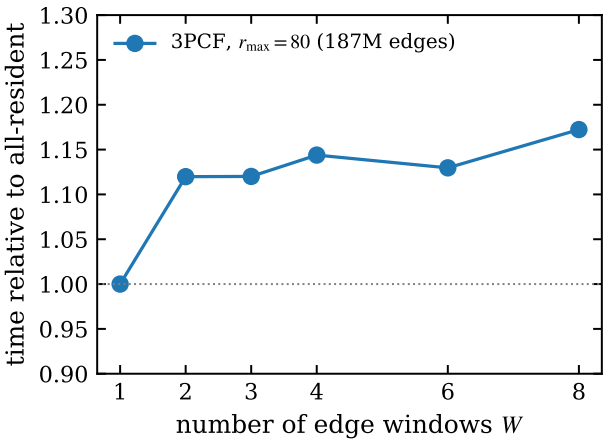

Figure 5. Measured out-of-core overhead: query time for a fixed problem forced through W edge windows, relative to the all-resident case. The one-time ∼12% cost saturates rather than growing with W.

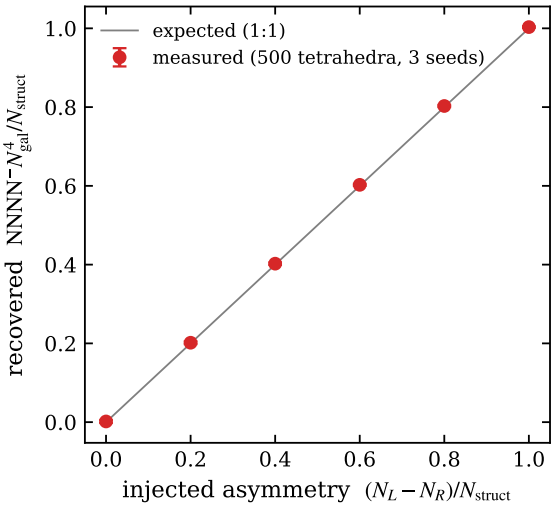

Figure 6. Signal-injection test of the parity-odd 4PCF. Catalogues of 500 isolated chiral tetrahedra with left-handed fraction x are measured with the GPU parity kernel; the count-normalised odd channel at the injected configuration recovers the injected asymmetry $2x-1$ (grey 1:1 line) to 0.3%, with the balanced case consistent with zero. Error bars show the scatter over three random realisations.

of Section 3.2 would be contaminated by floating-point-noise-signed tetrahedra; the passing null validates that treatment at the field level.

5. APPLICATION TO DESI DR1 LRG

As a realistic end-to-end application we measure the 2PCF, 3PCF, and connected 4PCF of the DESI DR1 LRG sample (DESI Collaboration & et al. 2025) (1.48M galaxies NGC, 0.66M SGC; weights $w \times w_{\rm FKP}$; DESI fiducial cosmology) with randoms subsampled 1:1, and compare against the EZmock ensemble (C.-H. Chuang et al. 2015) processed through the identical pipeline. Galactic caps are measured independently and combined at the level of raw counts (weighting each cap's normalised counts by the cube — or square, for pair counts — of its summed data weights). For the redshift-binned statistics we split at $z = 0.8$ (bins $0.4 < z \le 0.8$ and $0.8 < z \le 1.1$, with pair-weighted effective redshifts $z_{\rm eff} = 0.63$ and 0.96).

The combined $r^2\xi(r)$ shows the acoustic peak at $r \simeq 97.5\,h^{-1}$Mpc with the expected dip–peak structure, recovered independently in both caps. The full-sample 3PCF, measured to $r_{\max} = 150\,h^{-1}$Mpc in $5\,h^{-1}$Mpc bins (the 9.0×10^9-edge out-of-core measurement of Table 2), shows the corresponding BAO feature: in the isoceles slice the acoustic bump appears at $r_3 \simeq 107$–$118\,h^{-1}$Mpc, an order of magnitude above the surrounding baseline, with both Galactic caps showing the rise independently.

5.1. *3PCF versus the mock ensemble*

Figure 7 compares the redshift-binned DESI 3PCF with the EZmock ensemble in the isoceles slice $\zeta(55, 75, r_3)$, rebinned from 5 to $10\,h^{-1}$Mpc (exactly, since binned counts are additive). Each of the 25 mocks is processed through the same cap combination; the band shows the ensemble mean and scatter, and the same scatter provides the error bars on the data points. The agreement is good in both redshift bins ($\chi^2/\mathrm{dof} = 4.7/9$ and 9.8/9 against the mock mean, diagonal errors): in particular the negative-to-positive transition through the acoustic region seen in the data is reproduced by the mock mean, confirming that the BAO-scale three-point signal in the LRG sample is consistent with the ΛCDM-calibrated mocks.

5.2. *4PCF versus the mock ensemble*

[TODO: 4PCF comparison: DESI z-binned connected 4PCF (rmin=20, rmax=80, 4 bins, 276 configurations) vs the same 25 EZmocks — measurements running; add figure and chi2 when complete.]

6. VALIDATION ON N-BODY SIMULATIONS

6.1. *Setup*

We use Friends-of-Friends halo catalogues from the Quijote simulation suite (F. Villaescusa-Navarro & et al. 2020): 512^3 particles in $(1\,h^{-1}\mathrm{Gpc})^3$ boxes at $z = 0.5$ in the fiducial cosmology ($\Omega_{\rm m} = 0.3175$, $\sigma_8 = 0.834$, $h = 0.6711$). A number-density cut $\bar{n} = 1.5\times10^{-4}\,(h/\mathrm{Mpc})^3$ — the 150 000 most massive halos per box — is applied to each catalogue, with uniform randoms in the

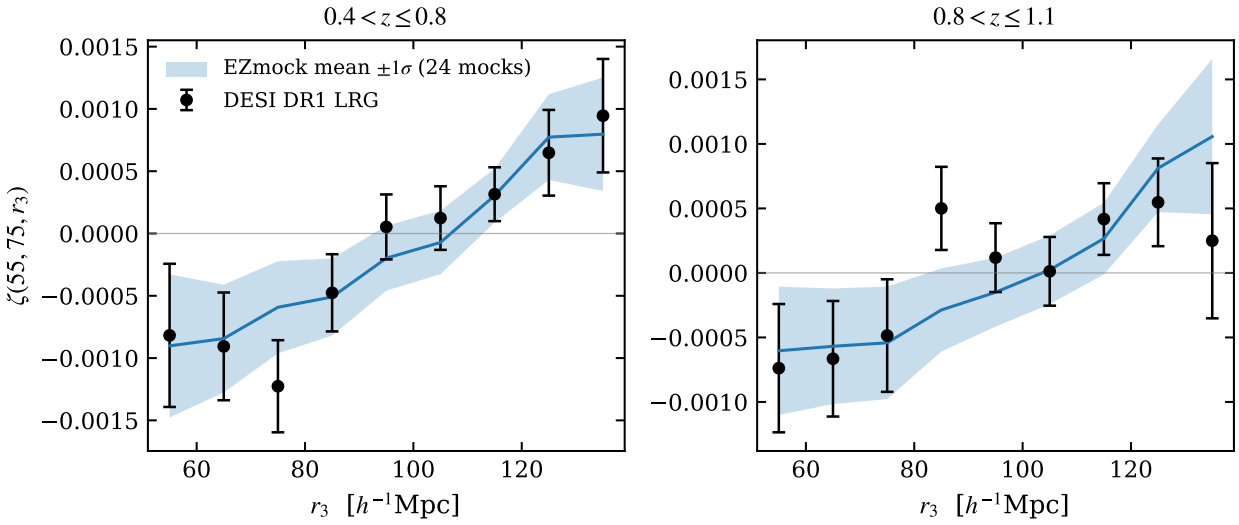

Figure 7. DESI DR1 LRG 3PCF versus the EZmock ensemble in the isoceles slice $\zeta(55, 75, r_3)$ ($10\,h^{-1}$Mpc bins), for the two redshift bins (NGC and SGC combined at the count level in both data and mocks). The shaded band is the mock ensemble mean $\pm1\sigma$ (25 realisations); data error bars use the same mock scatter. The acoustic-region rise in the data is reproduced by the ensemble.

periodic box at 1:1. We measure the 3PCF in 20 radial bins over 10–150 h^{-1}Mpc and the connected and parity-decomposed 4PCFs in 5 bins to 65 h^{-1}Mpc on 100 realisations, at ∼50 s per 3PCF and ∼9 s per 4PCF realisation on the single GPU.

6.2. *Fiducial measurement*

Figure 8 shows the fiducial 3PCF with box-to-box scatter along two one-dimensional cuts through the full configuration space: the equilateral cut and the isoceles slice $\zeta(48, 76, r_3)$ used for the DESI demonstration of Section 5. Both show the expected hierarchical behaviour at small scales, a shallow negative trough at intermediate separations, and amplitudes consistent with zero at large scales. Notably, no acoustic feature is visible at $r \approx 105\,h^{-1}$Mpc even in the 100-realisation mean: at tree level the 3PCF is built from products of two 2PCFs, so an equilateral configuration with all sides at the acoustic scale carries the feature only through $\xi(r_{\rm BAO})^2$ — quadratically suppressed to $\sim10^{-5}$, below even the ensemble sensitivity here. The acoustic signal in the 3PCF instead resides in isoceles configurations, $\zeta \sim \xi(r_{\rm short})\,\xi(r_{\rm BAO})$, which is the cut used for the DESI demonstration in Section 5 and the basis-compressed detections of Z. Slepian & et al. (2017). The GPU and CPU pipelines agree exactly on these catalogues (Section 4.8); the purpose of this ensemble is to exercise the full pipeline at production scale and to furnish the error bars and null tests used below and in Section 4.9.

7. SCIENTIFIC APPLICATIONS

Primordial non-Gaussianity—The connected 4PCF $\zeta_{\rm conn}$ is a direct configuration-space probe of the primordial trispectrum. The 9× GPU speedup at BAO-relevant scales makes it feasible to evaluate $\zeta_{\rm conn}$ across the full BAO range for DESI-scale catalogues, where covariance estimation requires $\mathcal{O}(10^3)$ mock realisations.

Parity violation—J. Hou et al. (2023) reported a non-zero parity-odd 4PCF in BOSS DR12 CMASS at 7.1σ using a spherical-harmonic estimator, with an independent analysis finding lower significance (O. H. E. Philcox 2022); the first DESI measurement has recently appeared (Z. Slepian & et al. 2025). The GRAMSCI configuration-space parity estimator provides a complementary cross-check that is sensitive to the full angular structure of the signal without assuming a harmonic decomposition. The deterministic treatment of degenerate tetrahedra (Section 3.2) is essential here: at the few-percent level the odd channel is otherwise contaminated by floating-point noise whose sign depends on the compiler and hardware.

BAO with higher-order statistics—GRAMSCI supports anisotropic 3PCF and 4PCF (via μ binning for redshift-space distortions), enabling joint 2PCF+3PCF+4PCF BAO fits that extract additional signal-to-noise from the same catalogue.

8. SUMMARY

We have presented four additions to GRAMSCI:

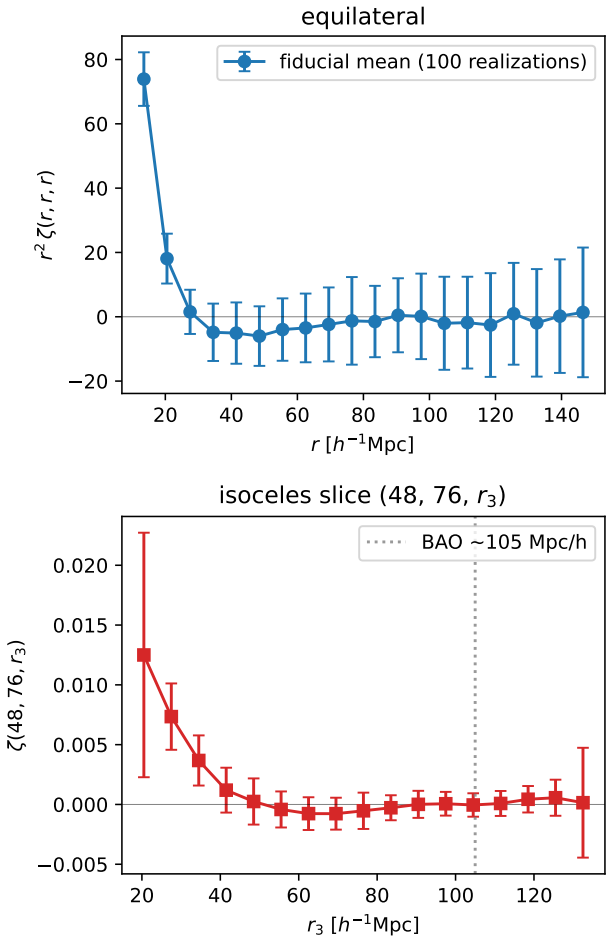

Figure 8. 3PCF of Quijote fiducial FoF halos at $z = 0.5$ (number density matched to $1.5 \times 10^{-4}\ (h/\mathrm{Mpc})^3$), averaged over 100 realisations: the equilateral cut (left) and the isoceles slice $\zeta(48, 76, r_3)$ (right). Error bars show the box-to-box scatter of a single $(1\,h^{-1}\mathrm{Gpc})^3$ realisation.

1. **Merge-walk inner kernel.** Replacing binary-search with two-pointer sorted-set intersection reduces the 3PCF inner loop from $O(m \log m)$ to $O(m)$ and the 4PCF inner loop — which required two binary searches — correspondingly further. Measured speedups are 1.5–2× for the 3PCF and 2.8–3.8× for the 4PCF at survey-relevant scales. The key prerequisite — adjacency lists sorted by node index — was already guaranteed by the KD-tree construction and required no data-structure change.

2. **Parity-decomposed 4PCF.** A direction-pixel byte stored per edge at graph-construction time allows the 4PCF to be decomposed into parity-even and parity-odd channels at negligible additional cost, enabling direct tests of parity violation. Degenerate (zero-volume) tetrahedra are excluded from the odd channel deterministically.

3. **Python interface.** A thin NumPy-based wrapper calls the Fortran engine via subprocess, handling all file I/O automatically and returning structured result objects.

4. **GPU port with out-of-core support.** The complete query engine runs on a single GPU via OpenACC with measured speedups of 2.6× (3PCF) to 9× (4PCF) over a 64-thread node, and an automatic window-tiling scheme processes graphs far larger than device memory (9×10^9 edges demonstrated on a 24 GB card). All kernels reproduce the CPU reference exactly.

The code is publicly available at https://github.com/csabiu/Gramsci.

Software: GRAMSCI (C. G. Sabiu et al. 2019); KDTREE2 (M. B. Kennel 2004); NumPy (C. R. Harris & et al. 2020); NVIDIA HPC SDK.

APPENDIX

A. OPENACC IMPLEMENTATION NOTES

We record four NVFORTRAN/OpenACC behaviours that silently destroy performance or correctness in this class of kernel; each cost significant debugging effort and none is prominent in the standard documentation.

Implicit gang–vector mapping—A bare `!$ACC PARALLEL LOOP GANG` over hubs lets the compiler map the hub loop across gangs *and* vector lanes, placing a different hub on every lane of a warp. Each lane then traverses a different adjacency list: memory access is fully divergent and warp execution time is set by the largest hub in the warp. An explicit inner `!$ACC LOOP VECTOR` over the neighbour loop, with the hub loop on gangs only, changed this kernel from 6× slower than the CPU node to 2.6× faster.

Private arrays are per-lane—Arrays in a `PRIVATE` clause of a gang loop with `VECTOR_LENGTH` > 1 are allocated per vector lane, not per gang. A 360 KB per-hub scratch array at vector length 64 implies 23 MB per thread block — far beyond the per-SM limit — and fails at kernel launch with `CUDA_ERROR_ILLEGAL_ADDRESS`. The workaround is a global scratch array indexed by an explicit gang-slot loop.

Array reductions—Reductions of multidimensional allocatable arrays do not reliably propagate device results back to the host, and array reductions declared on nested gang+vector loops re-reduce at the end of every inner-loop instance. Slot-strided partial arrays updated with `ATOMIC UPDATE` and summed on the host are both correct and fast.

By-reference scalar arguments go stale—Scalar arguments passed (by Fortran default) by reference into `!$ACC ROUTINE` device functions receive a one-shot device copy that is *not* refreshed on subsequent kernel launches: host-side updates between launches are silently ignored. In the out-of-core tiles this manifested as out-of-bounds reads from the second tile onward, while scalars used directly in kernel code (which become `firstprivate`) remained correct. All scalar arguments of device routines must carry the `VALUE` attribute.

REFERENCES

Anderson, L., & et al. 2014, MNRAS, 441, 24

Cahn, R. N., Slepian, Z., & Hou, J. 2023, Phys. Rev. Lett., 130, 201002

Chuang, C.-H., Kitaura, F.-S., Prada, F., Zhao, C., & Yepes, G. 2015, MNRAS, 446, 2621

DESI Collaboration, & et al. 2025, JCAP, 2025, 021

Gaztañaga, E., & Scoccimarro, R. 2005, MNRAS, 361, 824

Harris, C. R., & et al. 2020, Nature, 585, 357

Hou, J., Slepian, Z., & Cahn, R. N. 2023, MNRAS, 522, 5701

Jarvis, M., Bernstein, G., & Jain, B. 2004, MNRAS, 352, 338

Kennel, M. B. 2004, KDTREE2: Fortran 95 and C++ software to efficiently search for near neighbors in a multi-dimensional Euclidean space,

Landy, S. D., & Szalay, A. S. 1993, ApJ, 412, 64

Peebles, P. J. E., & Groth, E. J. 1975, ApJ, 196, 1

Philcox, O. H., & Slepian, Z. 2022, MNRAS, 509, 2457

Philcox, O. H. E. 2022, Phys. Rev. D, 106, 063501

Sabiu, C. G., Hoyle, B., Kim, J., & Li, X.-D. 2019, ApJS, 242, 29

Sefusatti, E., Crocce, M., Pueblas, S., & Scoccimarro, R. 2006, Phys. Rev. D, 74, 023522

Sefusatti, E., & Komatsu, E. 2007, Phys. Rev. D, 76, 083004

Sinha, M., & Garrison, L. H. 2020, MNRAS, 491, 3022

Slepian, Z., & Eisenstein, D. J. 2015, MNRAS, 454, 4142

Slepian, Z., & Eisenstein, D. J. 2016, MNRAS, 455, L31

Slepian, Z., & et al. 2017, MNRAS, 469, 1738

Slepian, Z., & et al. 2025, arXiv e-prints

Szapudi, I., & Szalay, A. S. 1998, ApJ, 494, L41

Villaescusa-Navarro, F., & et al. 2020, ApJS, 250, 2